

PaymentProcessor.java - Deep Dive Explanation

1. OVERVIEW & LINKAGE

This is the Execution Engine. It links to the Payment abstract class by maintaining a queue (List<Payment>). Because it only talks to the generic Payment class, it is completely decoupled from the specific child classes. This means you could add 50 new payment types in the future, and this file would never need to change.

2. DETAILED CODE BREAKDOWN

```
private List<Payment> queue = new ArrayList<>();
```

This list holds pending transactions. Notice it holds "Payment", not "CreditCardPayment". This is polymorphic storage. Because all payment types inherit from Payment, they can all sit in this single queue together.

```
public List<ProcessResult> processAll() { ... }
```

This is the main algorithm where dynamic dispatch happens. It creates an empty list of results, then begins a "for" loop to iterate over every pending transaction in the queue.

```
BiConsumer<String, String> log = (text, kind) -> lines.add(new String[]{ text, kind });
```

This creates a lambda function (an anonymous method). We pass this function down into the payment.process() method so the payment can send log messages back up to the processor.

```
boolean success = payment.process(log);
```

THIS IS THE MAGIC LINE. Notice there are no "if" statements checking if it is a credit card or PayPal. The Java Virtual Machine (JVM) looks at the generic "payment" variable in memory, figures out what child class it actually was instantiated as, and dynamically runs that specific class's overridden process() method.

```
public double totalSuccessful() { ... }
```

This algorithm uses Java 8 Streams to loop through the history list, filter out anything whose status is not "SUCCESS", extract the amount of the successful transactions, and sum them up mathematically.